

ZYVORAI LABS · CUSTOMER DOCUMENTATION



Page-by-Page Product Manual

2026-08-29

Page-by-page guides

Each guide follows: Purpose → When to use it → How to get there → Operate from CLI → Related pages.

Every route is also listed in the [complete page index](#).

Cli

Page	What it covers
CLI: events	Tail normalized carrier, routable, routes, and manager events from the daemon API — the same stream hooks react to, but for operators debugging link flaps or missing scripts.
CLI: list / show	Inspect interfaces, routes, policy rules, and registered hook scripts — the same objects netevd tracks for hook dispatch and automatic policy routing.
CLI: status	Show daemon health, uptime, interface count, routing-rule count, and events processed — a single glance before you change hooks or YAML.
CLI: validate / reload / test	Validate YAML offline, request a config reload through the API, and dry-run hook scripts — the safe path before touching production networking.

Hooks

Page	What it covers
Hook: carrier / no-carrier	Run scripts when the link layer goes UP (<code>carrier.d</code>) or DOWN (<code>no-carrier.d</code>) — cable plug, Wi-Fi associate, or switch port flap — before L3 addresses exist.
Hook: manager / NetworkManager	React to NetworkManager device and manager state via <code>activated.d</code> , <code>disconnected.d</code> , and <code>manager.d</code> — the NM equivalent of systemd-networkd's configured/routable hooks.
Hook: routable	Fire when an interface has full L3 connectivity — addresses and a usable path — on systemd-networkd and dhclient backends. This is the most common hook directory for “start my service when the network is actually up.”
Hook: routes	React to routing-table changes via <code>/etc/netevd/routes.d/</code> — default-route moves, VPN injection, or policy-table updates that netevd detects through its route watcher.

Ops

Page	What it covers
Configuration YAML	Tune backend, monitored interfaces, policy routing, API/metrics bind, event filters, and audit logging — the control plane for every hook and CLI query.
Prometheus metrics (:9091)	Scrape hook execution, event throughput, routing-rule counts, and netlink health for fleet observability — complementing REST status checks on <code>:9090</code> .
REST API (:9090)	Query status, interfaces, routes, policy rules, and recent events over HTTP — the same data backing <code>netevd status</code> , <code>list</code> , <code>show</code> , and <code>events</code> without SSH or <code>ip(8)</code> .

11 guides. Regenerate: `node scripts/customer-docs/generate-guide-index.mjs` .

CLI: events

Purpose

Tail normalized carrier, routable, routes, and manager events from the daemon API — the same stream hooks react to, but for operators debugging link flaps or missing scripts.

When to use it

- A hook did not fire and you need to see whether netevd saw the event
- You are validating filters in `/etc/netevd/netevd.yaml` while bouncing an interface
- You want a live feed during a change window instead of parsing `journalctl`

How to get there

- Command: `netevd events`
- Default API: `http://127.0.0.1:9090` (override with `--endpoint http://<host>:9090`)
- Requires `api.enabled: true` in `/etc/netevd/netevd.yaml` and a running `netevd.service`

Operate from CLI

1. Confirm the daemon and API are up:

```
systemctl is-active netevd
curl -sI http://127.0.0.1:9090/health | jq .
```

2. Show the last 20 events (text table):

```
netevd events --tail 20
```

3. Follow live events (like `tail -f`):

```
netevd events -f
```

4. Filter by interface and event type while testing a link bounce:

```
netevd events -f -i eth0 -t routable
```

5. Machine-readable output for automation:

```
netevd events --tail 50 -f json --endpoint http://<host>:9090
```

6. Cross-check hook side effects in the journal:

```
journalctl -u netevd -f
```

7. **Empty / fail:** API unreachable → check `api.enabled`, bind address, and firewall; events list empty → confirm `system.backend` matches the host (`systemd-networkd`, `NetworkManager`, or `dhclient`) and the interface is not ignored by a YAML filter.

8. **Success:** Events appear within ~100 ms of a link or address change; filtered `-i / -t` flags narrow the stream to the interface under test.

Related pages

- [CLI: status](#)
 - [Hook: routable](#)
 - [REST API](#)
 - [Common workflows](#)
-

CLI: list / show

Purpose

Inspect interfaces, routes, policy rules, and registered hook scripts — the same objects netevd tracks for hook dispatch and automatic policy routing.

When to use it

- Multi-homed hosts: confirm per-interface routing tables after `routing.policy_rules` changes
- Debugging: compare `netevd list rules` with `ip rule list`
- Inventory: see which hook scripts netevd knows about before a rollout

How to get there

- Commands: `netevd list ...`, `netevd show ...`
- Default API: `http://127.0.0.1:9090` (`--endpoint` for remote hosts)
- Config reference: `/etc/netevd/netevd.yaml`

Operate from CLI

1. List tracked interfaces:

```
netevd list interfaces
netevd list interfaces -f json
```

2. List kernel routes netevd is aware of:

```
netevd list routes
```

3. List policy routing rules (multi-homed verification):

```
netevd list rules
ip rule list
ip route show table $((200 + $(cat /sys/class/net/eth1/ifindex)))
```

4. List hook scripts discovered under `/etc/netevd/*.d/` :

```
netevd list scripts
```

5. Drill into one interface:

```
netevd show interface eth1
netevd show interface eth0 -f yaml --endpoint http://<host>:9090
```

6. **Empty / fail:** Lists empty but links exist → widen `monitoring.interfaces` (empty = all) or remove an overly broad `filters` ignore rule; API errors → see [CLI: status](#).
7. **Success:** Interface count matches expectations; `list rules` shows `from / to` pairs for each address on policy-routed interfaces.

Related pages

- [CLI: events](#)
 - [Hook: routes](#)
 - [Configuration YAML](#)
 - [Multi-homed workflow](#)
-

CLI: status

Purpose

Show daemon health, uptime, interface count, routing-rule count, and events processed — a single glance before you change hooks or YAML.

When to use it

- Post-install smoke test on a new host
- Change-window entry: confirm netevd is running before editing `/etc/netevd/`
- Fleet checks from cron or a monitoring probe (JSON output)

How to get there

- Command: `netevd status`
- Equivalent HTTP: `GET http://127.0.0.1:9090/api/v1/status`
- Unit: `systemctl status netevd`

Operate from CLI

1. Quick text summary (default):

```
netevd status
```

2. JSON for scripts and dashboards:

```
netevd status -f json
netevd status -f json --endpoint http://<host>:9090 | jq '.data'
```

3. YAML output:

```
netevd status -f yaml
```

4. Cross-check systemd and health endpoint:

```
systemctl status netevd --no-page
url -sf http://127.0.0.1:9090/health | jq .
url -sf http://127.0.0.1:9090/api/v1/status | jq .
```

5. **Empty / fail:** `Connection refused` → daemon down or `api.enabled: false` ; status shows zero interfaces on a busy host → backend mismatch or netlink permission issue (check unit logs).
6. **Success:** `status: running` , non-zero uptime, interface count matches `ip link` , routing-rule count matches policy-routed addresses.

Related pages

- [REST API](#)
- [Prometheus metrics](#)
- [Admin basics](#)

CLI: validate / reload / test

Purpose

Validate YAML offline, request a config reload through the API, and dry-run hook scripts — the safe path before touching production networking.

When to use it

- Before every edit to `/etc/netevd/netevd.yaml`
- After adding or renaming hook scripts (validate + restart/reload)
- When developing a new hook: preview environment variables without firing real events

How to get there

- Commands: `netevd validate`, `netevd reload`, `netevd test`, `netevd version`
- Config path: `/etc/netevd/netevd.yaml` (override with `-c` / `--config`)
- Global flags on the daemon binary: `--dry-run`, `-v` (verbose)

Operate from CLI

1. Validate the live config (default path):

```
sudo netevd validate
```

2. Validate a staged file before install:

```
netevd validate -c /tmp/netevd.yaml
```

3. Request reload via API (preferred when the daemon supports hot reload):

```
netevd reload --endpoint http://127.0.0.1:9090  
curl -X POST http://127.0.0.1:9090/api/v1/reload -H 'Content-type: application/json' -d '{}'
```

4. When reload is not yet applied server-side, restart the unit (config + hook directory changes):

```
sudo systemctl restart netevd  
journalctl -u netevd -n 30 --no-pager
```

5. Dry-run the daemon (no script execution, no routing mutations):

```
sudo netevd --dry-run -c /etc/netevd/netevd.yaml start
```

6. Preview hook environment for a script under test:

```
netevd test /etc/netevd/routable.d/01-notify.sh -i eth0 -t routable --ip 192.168.1.10
```

7. Print build/feature matrix:

```
netevd version --detailed
```

8. **Empty / fail:** Validation errors name the YAML key; reload returns `success: false` → fall back to `systemctl restart netevd` ; test mode prints env but does not execute — confirm with a real link bounce + `journalctl` .
9. **Success:** ✓ `Configuration is valid` summary lists backend, interfaces, and policy rules; daemon restarts cleanly; test output shows expected `$LINK` , `$STATE` , `$ADDRESSES` .

Related pages

- [Configuration YAML](#)
 - [Admin basics](#)
 - [CLI: events](#)
-

Hook: carrier / no-carrier

Purpose

Run scripts when the link layer goes UP (`carrier.d`) or DOWN (`no-carrier.d`) — cable plug, Wi-Fi associate, or switch port flap — before L3 addresses exist.

When to use it

- Notify ops when a physical link returns (PDU cycle, switch maintenance)
- Trigger low-level diagnostics that do not need an IP yet
- **Not** for “has an IP / default route” — use [Hook: routable](#) for L3

How to get there

- Directories: `/etc/netevd/carrier.d/` , `/etc/netevd/no-carrier.d/`
- Backends: all (`systemd-networkd` , `NetworkManager` , `dhclient`)
- Scripts must be executable; run in lexical order (`10-` , `20-` , ...)

Operate from CLI

1. Install a carrier-up logger:

```
sudo tee /etc/netevd/carrier.d/10-link-up.sh >/dev/null <<'EOF'
#!/bin/bash
logger -t netevd "carrier UP on $LINK (index $LINKINDEX, backend $BACKEND)"
EOF
sudo chmod +x /etc/netevd/carrier.d/10-link-up.sh
```

2. Install a no-carrier handler:

```
sudo tee /etc/netevd/no-carrier.d/10-link-down.sh >/dev/null <<'EOF'
#!/bin/bash
logger -t netevd "carrier DOWN on $LINK"
EOF
sudo chmod +x /etc/netevd/no-carrier.d/10-link-down.sh
```

3. Validate config and bounce the link to test:

```
netevd validate
sudo ip link set eth0 down && sudo ip link set eth0 up
netevd events -i -i eth0 -t carrier &
journalctl -u netevd -f
```

4. Dry-run hook env without executing:

```
netevd test /etc/netevd/carrier.d/10-link-up.sh -i eth0 -t carrier
```

5. **Empty / fail:** Script never runs → missing `+x` , wrong directory, or YAML filter with `action: ignore` on `docker*` /pattern match; check `journalctl -u netevd` for exit codes (non-zero exits are logged, other

scripts still run).

6. **Success:** `logger` or script output appears on link up/down; `$LINK` and `$BACKEND` are set; carrier events visible in `netevd events` .

Related pages

- [Hook: routable](#)
 - [CLI: events](#)
 - [Admin basics](#)
-

Hook: manager / NetworkManager

Purpose

React to NetworkManager device and manager state via `activated.d`, `disconnected.d`, and `manager.d` — the NM equivalent of systemd-networkd's configured/routable hooks.

When to use it

- Hosts where NetworkManager is primary (`system.backend: NetworkManager`)
- VPN or Wi-Fi profiles that activate/disconnect without classic netlink carrier semantics
- Manager-wide scripts (restart dependent services when NM reconnects)

How to get there

- Directories: `/etc/netevd/activated.d/`, `/etc/netevd/disconnected.d/`, `/etc/netevd/manager.d/`
- Config: set `system.backend: NetworkManager` in `/etc/netevd/netevd.yaml`
- Requires NM D-Bus signals; unit starts after `NetworkManager.service`

Operate from CLI

1. Confirm backend and restart:

```
grep backend /etc/netevd/netevd.yaml
netevd validate
sudo systemctl restart netevd
```

2. Add an activation hook:

```
sudo tee /etc/netevd/activated.d/20-nm-up.sh >/dev/null <<'EOF'
#!/bin/bash
logger -t netevd "NM activated $LINK state=$STATE addrs=$ADDRESSES"
00
sudo chmod +x /etc/netevd/activated.d/20-nm-up.sh
```

3. Add a disconnect hook:

```
sudo tee /etc/netevd/disconnected.d/10-nm-down.sh >/dev/null <<'EOF'
#!/bin/bash
logger -t netevd "NM disconnected $LINK"
00
sudo chmod +x /etc/netevd/disconnected.d/10-nm-down.sh
```

4. Manager-level script (all interfaces / manager events):

```
sudo install -m755 ./my-manager-hook.sh /etc/netevd/manager.d/05-manager.sh
```

5. Test with NM CLI while tailing events:

```
netevd events -f -t activated &
```

```
nmcli connection down my-wifi && nmcli connection up my-wifi
```

```
journalctl -u netevd -n 20 --no-pager
```

6. **Empty / fail:** Hooks silent on NM host → backend still `systemd-networkd` ; NM device state changes but no scripts → check filters for `backend: NetworkManager` with `action: log` only.
7. **Success:** Activation/disconnect logged; `$STATE` reflects NM normalized state; events appear in `netevd events -t activated` .

Related pages

- [Configuration YAML](#)
 - [Hook: routable](#)
 - [Admin basics](#)
-

Hook: routable

Purpose

Fire when an interface has full L3 connectivity — addresses and a usable path — on systemd-networkd and dhclient backends. This is the most common hook directory for “start my service when the network is actually up.”

When to use it

- Register DNS, announce presence, or start app stacks when `$ADDRESSES` is meaningful
- DHCP renewals on edge nodes (dhclient path)
- Consume rich `$JSON` from systemd-networkd (MTU, DNS, routes) when `emit_json: true`

How to get there

- Directory: `/etc/netevd/routable.d/`
- Backends: `systemd-networkd`, `dhclient` (not NM — use [Hook: manager](#))
- Example env: `$LINK`, `$ADDRESSES`, `$JSON` (networkd), `$DHCP_*` (dhclient)

Operate from CLI

1. Install the canonical first hook from [Getting Started](#):

```
sudo tee /etc/netevd/routable.d/01-notify.sh >/dev/null <<'EOF'
#!/bin/bash

logger -t netevd "Interface $LINK is routable: $ADDRESSES (backend=$BACKEND)"

01
sudo chmod +x /etc/netevd/routable.d/01-notify.sh
```

2. Enable JSON payload for networkd scripts in YAML:

```
sudo sed -i "/systemd-networkd/,/networkmanager/p" /etc/netevd/netevd.yaml
ensure: emit_json: true under backends.systemd networkd
netevd validate && sudo systemctl restart netevd
```

3. Trigger a routable event (networkd host):

```
sudo networkctl renew eth0 # or reapply your .network unit
netevd events -f -I eth0 -t routable
```

4. dhclient host — renew lease:

```
sudo dhclient -r eth0 && sudo dhclient eth0
eth0 "CHECK \${DHCP_ADDRESS} \${DHCP_GATEWAY} in journal"
```

5. Inspect script inventory:

```
netevd list scripts
```

```
ls -la /etc/netevd/routable.d/
```

6. **Empty / fail:** Hook runs on carrier but not routable → script is in wrong `*.d` dir; `$JSON` empty → `emit_json: false`; dhclient vars missing → event fired before lease parse completes.
7. **Success:** `logger` shows addresses; dependent service starts; `netevd_script_executions_total` increments (see [Prometheus](#)).

Related pages

- [Hook: carrier](#)
 - [Common workflows](#)
 - [CLI: validate / reload](#)
-

Hook: routes

Purpose

React to routing-table changes via `/etc/netevd/routes.d/` — default-route moves, VPN injection, or policy-table updates that netevd detects through its route watcher.

When to use it

- Audit when the default gateway shifts between uplinks
- Side effects after netevd installs per-interface policy tables (see [Multi-homed workflow](#))
- Integrate with SD-WAN or VPN clients that mutate `ip route` frequently

How to get there

- Directory: `/etc/netevd/routes.d/`
- Backends: all
- Complements automatic policy routing from `routing.policy_rules` in YAML

Operate from CLI

1. Install a route-change auditor:

```
sudo tee /etc/netevd/routes.d/30-audit-routes.sh >/dev/null <<'EOF'
#!/bin/bash
logger -t netevd "route change on $LINK state=$STATE backend=$BACKEND"
exit 0
sudo chmod +x /etc/netevd/routes.d/30-audit-routes.sh
```

2. Enable policy routing for a secondary uplink in YAML:

```
grep -A5 '^routing:' /etc/netevd/netevd.yaml
netevd validate
sudo systemctl restart netevd
```

3. Verify rules and routes after address acquisition:

```
netevd list rules
netevd list routes
ip route show table all | head -30
```

4. Watch route events during a controlled change:

```
netevd events -f -t routes &
sudo ip route replace default via 192.168.1.1 dev eth1 # lab only
```

5. **Empty / fail:** No route hooks despite table changes → script not executable; route events filtered; compare with `journalctl -u netevd` for netlink errors.
6. **Success:** Hook logs on table mutation; `list rules` reflects policy tables; traffic sourced from secondary IP uses correct table (`ip rule list`).

Related pages

- [CLI: list / show](#)
 - [Configuration YAML](#)
 - [Multi-homed workflow](#)
-

Configuration YAML

Purpose

Tune backend, monitored interfaces, policy routing, API/metrics bind, event filters, and audit logging — the control plane for every hook and CLI query.

When to use it

- Initial install: copy from `config/netevd.example.yaml`
- Multi-homed servers: list interfaces under `routing.policy_rules`
- Noisy lab hosts: add `filters` to ignore `docker*`, `veth*`, `br-*`
- Fleet scrape: enable metrics and optionally widen API bind on trusted networks

How to get there

- Live file: `/etc/netevd/netevd.yaml`
- Template: `config/netevd.example.yaml` in the repo or release tarball
- Validate before apply: `netevd validate`

Operate from CLI

1. Copy the example and edit:

```
sudo install -Dm644 config/netevd.example.yaml /etc/netevd/netevd.yaml
sudo ${EDITOR:-vi} /etc/netevd/netevd.yaml
```

2. Set backend to match the host:

```
systemd
backend: "systemd-networkd" # or NetworkManager | dhclient
```

3. Limit monitoring scope (empty list = all interfaces):

```
monitoring:
interfaces:
  - eth0
  - eth1
```

4. Enable multi-homed policy routing:

```
routing:
policy_rules:
  - eth1
```

5. Open API/metrics for fleet hosts (trusted network only):

```
api:
enabled: true
```

```
bind address: "127.0.0.1" # use 0.0.0.0 only on isolated management nets
port: 9090
metrics:
  enabled: true
  port: 9091
```

6. Add a filter to ignore container interfaces:

```
filters:
  - match_rule:
      interface_pattern: "docker*"
    action: ignore
```

7. Validate, apply, verify:

```
neteyd validate -c /etc/neteyd/neteyd.yaml
sudo systemctl restart neteyd
neteyd status -f json | jq .
journalctl -u neteyd -n 20 --no-pager
```

8. **Empty / fail:** Parse error → validator prints the offending key; hooks stop firing → check filter `action: ignore` ; API unreachable → `api.enabled: false` or bind still `127.0.0.1` on remote scrape.

9. **Success:** Summary from `neteyd validate` matches intent; daemon logs show chosen backend; hooks and `list rules` behave per YAML.

Related pages

- [CLI: validate / reload](#)
 - [Admin basics](#)
 - [REST API](#)
-

Prometheus metrics (:9091)

Purpose

Scrape hook execution, event throughput, routing-rule counts, and netlink health for fleet observability — complementing REST status checks on `:9090`.

When to use it

- Alert when `netevd_script_failures_total` rises after a hook deploy
- Dashboard event rates per interface/backend
- Detect silent daemon failure (uptime gauge flatlines)

How to get there

- Metrics port: `metrics.port` in `/etc/netevd/netevd.yaml` (default **9091**)
- Enable: `metrics.enabled: true`
- Endpoint: `http://<host>:9091/metrics` (Prometheus text exposition)
- Note: `/metrics` also exists on the API router at `:9090` when API is enabled; prefer the dedicated metrics port for scrape jobs

Operate from CLI

1. Confirm metrics are enabled:

```
grep -A3 '^metrics:' /etc/netevd/netevd.yaml
curl -sf http://127.0.0.1:9091/metrics | head -20
```

2. Inspect key series locally:

```
curl -sf http://127.0.0.1:9091/metrics | grep -E '^netevd (info|events_total|script_|routing_rules_)
```

3. Prometheus scrape config (replace `<host>` with your edge/management target):

```
scrape_configs:
  - job_name: netevd
    scrape_interval: 30s
    static_configs:
      - targets: [ <host>:9091 ]
    labels:
      role: network-events
```

4. Example alert rules (hook failures):

```
groups:
  - name: netevd
    rules:
      - alert: NetevdHookFailures
        expr: increase(netevd_script_failures_total[5m]) >
        for: 2m
```

```
labels:
severity: warning
- alert: NetevdDown
expr: absent(netevd_info)
for: 5m
labels:
severity: critical
```

5. Remote check from your workstation:

```
curl -sI http://<host>:9091/metrics | grep netevd_uptime_seconds
```

6. **Empty / fail:** Connection refused → `metrics.enabled: false` or firewall; series missing → daemon just started or metrics init failed (see `journalctl -u netevd`).
7. **Success:** `netevd_info{version="..."}` present; counters increment when you bounce a link and routable hooks run.

Related pages

- [REST API](#)
 - [CLI: status](#)
 - [Fleet workflow](#)
-

REST API (:9090)

Purpose

Query status, interfaces, routes, policy rules, and recent events over HTTP — the same data backing `netevd status`, `list`, `show`, and `events` without SSH or `ip(8)`.

When to use it

- Automation and config-management probes from a bastion
- Quick inspection when CLI is not installed on a jump host (curl only)
- Integrate with internal dashboards (JSON responses)

How to get there

- Base URL: `http://127.0.0.1:9090` (default bind **localhost**)
- Config: `api.enabled`, `api.bind_address`, `api.port` in `/etc/netevd/netevd.yaml`
- Remote access: tunnel or set `bind_address: "0.0.0.0"` only on trusted management networks

Operate from CLI

1. Health and version smoke test:

```
curl -sf http://127.0.0.1:9090/health | jq
curl -sf http://127.0.0.1:9090/api/v1/status | jq .
```

2. List interfaces and drill into one:

```
curl -sf http://127.0.0.1:9090/api/v1/interfaces | jq
curl -sf http://127.0.0.1:9090/api/v1/interfaces/eth0 | jq .
```

3. Routes and policy rules:

```
curl -sf http://127.0.0.1:9090/api/v1/routes | jq
curl -sf http://127.0.0.1:9090/api/v1/rules | jq .
```

4. Recent events:

```
curl -sf 'http://127.0.0.1:9090/api/v1/events' | jq .
```

5. Request config reload (same as `netevd reload`):

```
curl -X POST http://127.0.0.1:9090/api/v1/reload \
-H 'Content-Type: application/json' \
-d '{"force":false}' | jq .
```

6. Remote host via SSH tunnel (keeps API on localhost):

```
ssh -L 9090:127.0.0.1:9090 root@host
```

```
curl -sf http://127.0.0.1:9090/api/v1/status | jq .
```

7. CLI equivalents:

```
netevd status -f json
```

```
netevd list interfaces -f json
```

```
netevd events --tail 20 -f json
```

8. **Empty / fail:** `Connection refused` → unit down or `api.enabled: false` ; empty event history → normal on fresh install until links change; reload returns `success: false` → restart with `systemctl restart netevd` .

9. **Success:** JSON `success: true` with populated `data` ; interface count matches `netevd list interfaces` ; health reflects dbus/netlink/config checks.

Endpoint reference

Method	Path	Description
GET	<code>/health</code>	Liveness and subsystem checks
GET	<code>/metrics</code>	Prometheus text (also on <code>:9091</code>)
GET	<code>/api/v1/status</code>	Daemon summary
GET	<code>/api/v1/interfaces</code>	All interfaces
GET	<code>/api/v1/interfaces/{name}</code>	One interface
GET	<code>/api/v1/routes</code>	Routing table
GET	<code>/api/v1/rules</code>	Policy rules
GET	<code>/api/v1/events</code>	Event history
POST	<code>/api/v1/reload</code>	Config reload request

Related pages

- [CLI: status](#)
 - [CLI: list / show](#)
 - [Prometheus metrics](#)
-

netevd — Complete page index

Every primary navigable dashboard route.

Generated: 2026-08-29 · 11 routes

Regenerate: `node scripts/customer-docs/generate-page-index.mjs`

CLI

Page	Route	Purpose	Guide
CLI: events	<code>/events</code>	Tail normalized carrier, routable, routes, and manager events from the daemon API.	Open
CLI: list / show	<code>/list-show</code>	Inspect interfaces, routes, policy rules, and registered hook scripts.	Open
CLI: status	<code>/status</code>	Show daemon health, uptime, and counters via the REST API.	Open
CLI: validate / reload / test	<code>/validate-reload</code>	Validate YAML offline; reload config or dry-run hook scripts safely.	Open

Hooks

Page	Route	Purpose	Guide
Hook: carrier / no-carrier	<code>/carrier</code>	Run scripts when link layer goes UP (carrier.d) or DOWN (no-carrier.d).	Open
Hook: manager / NetworkManager	<code>/manager</code>	React to NetworkManager device activation, disconnect, and manager state.	Open
Hook: routable	<code>/routable</code>	Fire when an interface has full L3 connectivity (systemd-networkd or dhclient).	Open
Hook: routes	<code>/routes</code>	React to routing-table changes via routes.d hooks.	Open

Ops

Page	Route	Purpose	Guide
Configuration YAML	<code>/config-yaml</code>	Tune backend, monitored interfaces, policy routing, API/metrics, filters, and audit.	Open
Prometheus metrics (:9091)	<code>/prometheus</code>	Scrape hook and event health metrics for fleet observability.	Open
REST API (:9090)	<code>/rest-api</code>	Query status, interfaces, routes, rules, and events without SSH or ip(8).	Open

Related

- [Customer docs home](#)
- [Page-by-page guides](#)